

SMART EXPENSE TRACKER

Interview Question Bank (Beginner, Intermediate, Expert)

Use case: MERN stack interview prep based on this codebase.

BEGINNER LEVEL (10)

- 1) Q: What frontend and backend stack is used in this project?
A: Frontend uses React + Vite + Tailwind CSS. Backend uses Node.js + Express + MongoDB.
- 2) Q: What is the backend entry point?
A: backend/src/server.js initializes environment, connects DB, and starts Express server.
- 3) Q: Where are API routes mounted?
A: backend/src/app.js mounts routes under /api/auth, /api/expenses, /api/budget, /api/analytics, /api/income, /api/reset.
- 4) Q: Which token mechanism is used for authentication?
A: JWT (jsonwebtoken).
- 5) Q: Which library is used for password hashing?
A: bcryptjs.
- 6) Q: How are protected routes handled on frontend?
A: PrivateRoute in frontend/src/App.jsx checks user/demo/token and redirects to /login if unauthorized.
- 7) Q: How is API base URL set in frontend?
A: frontend/src/utils/axios.js uses VITE_API_URL with fallback http://localhost:3000.
- 8) Q: What are key fields in Expense model?
A: user, title, amount are required; category defaults to Other; date defaults to Date.now.
- 9) Q: Which context handles budget state in frontend?
A: BudgetContext (frontend/src/context/BudgetContext.jsx).
- 10) Q: What is demo mode in this app?
A: A localStorage-driven mode where frontend contexts use demo data without real API persistence.

INTERMEDIATE LEVEL (10)

- 11) Q: How is dynamic CORS configured?
A: CORS_ORIGINS env is parsed into allowed origins; '*' allows all; other origins are checked via callback.
- 12) Q: Why might auth/reset routes be skipped by rate limiter?
A: To avoid blocking critical login/reset flows due to aggressive throttling.
- 13) Q: What does auth middleware attach after token verification?
A: req.user = User.findById(decoded.id).select('-password').
- 14) Q: Why use nullish coalescing (??) during updates?

- A: It preserves current values when incoming fields are null/undefined.
- 15) Q: What is budget rollover logic here?
A: Current month budget can carry forward unspent amount from previous month when rollover is enabled.
- 16) Q: What is effectiveLimit?
A: effectiveLimit = limit + carriedOver.
- 17) Q: How is category analytics computed?
A: Mongo aggregation pipeline using \$match and \$group by category.
- 18) Q: How is auth token auto-attached in API calls?
A: Axios request interceptor injects Authorization: Bearer <token>.
- 19) Q: What happens on HTTP 401 in frontend interceptor?
A: Token is removed from localStorage and user is redirected to /login.
- 20) Q: How does export feature work?
A: CSV via Blob download; printable report via new window + HTML table + window.print().

=====

EXPERT / INTERVIEW LEVEL (10)

=====

- 21) Q: What subtle middleware bug can happen in protect()?
A: Missing return after res.status(...).json(...) may allow fall-through and cause multi-response issues.
- 22) Q: Why is inconsistent use of req.user vs req.user.id risky?
A: It weakens consistency/type safety and can create edge-case bugs across controllers.
- 23) Q: What is a production security concern in DB config?
A: Mongo URI is logged in console (secret leakage risk).
- 24) Q: What schema-level data integrity issue exists in Budget model?
A: No unique compound constraint for (user, month, year), allowing duplicates.
- 25) Q: What analytics correctness issue exists in category breakdown?
A: It is all-time aggregation; without date filter UI can misrepresent monthly insights.
- 26) Q: What tradeoff exists in AuthContext state design?
A: It stores only token, not full user profile, so role/profile checks need extra fetch.
- 27) Q: What security implication does demo mode have?
A: localStorage flags are client-controlled; UI trust is weak (backend protection still required).
- 28) Q: What scaling issue may occur in budget history?
A: Unbounded growth of embedded history array can bloat document size.
- 29) Q: What UX issue can appear with global 401 redirect interceptor?
A: Concurrent 401 responses can trigger repeated redirects/logout race.
- 30) Q: Name 3 immediate production hardening tasks.
A: (1) Remove secret logs + enforce strong JWT secret policy
(2) Add request validation for all write endpoints

r (3) Add indexes/constraints + automated tests for auth and budget rollover

=====
END OF DOCUMENT
=====